BỘ GIÁO DỤC & ĐÀO TẠO

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG-HCM

KHOA CÔNG NGHỆ THÔNG TIN

Lab 4 – Cryptocurrencies Prices

Môn học: Nhập môn dữ liệu lớn

Bộ môn: Khoa học dữ liệu

Mã lớp: CQ2022/21

GVHD: Nguyễn Ngọc Thảo

Tên nhóm: CQ06

Thành viên:

Nguyễn Đình Trí	MSSV: 22120384
Vũ Hoàng Nhật Trường	MSSV: 22120398
Nguyễn Anh Tường	MSSV: 22120412
Nguyễn Hoàng Vũ	MSSV: 22120439

Thành phố Hồ Chí Minh, ngày 10 tháng 06 năm 2025

Mục lục

1. Phân công thành viên nhóm 1
2. Setup environment 2
3. Extraction 4
3.1. Đảm bảo tần suất 4
3.1.1. Độ phức tạp khi code 4
3.1.2. Hiệu suất 6
3.1.3. Tiêu tốn tài nguyên 7
3.2. Chi tiết cách ingest dữ liệu vào kafka 8
3.3. Kết quả 10
4. Transformation 11
4.1. Moving average 11
4.2. Zscore 19
4.3. Kết quả 22
5. Loading 23
5.1. Schema lựa chọn 23
5.2. Cách làm 23
5.3. Kết quả 25

1. Phân công thành viên nhóm

MSSV	Họ và tên	Phân công công việc
22120384	Nguyễn Đình Trí	❖ Setup Environment
22120398	Vũ Hoàng Nhật Trường	❖ Loading
22120412	Nguyễn Anh Tường	❖ Extraction
22120439	Nguyễn Hoàng Vũ	❖ Transformation ❖ Report

2. Setup environment

Do muốn làm quen với một công cụ đóng gói mạnh mẽ là docker nên em quyết định làm lab này trên docker và chọn hệ điều hành linux để chạy nên có thể còn một số lỗi sai sót trên môi trường window.

```yaml
Run All Services
services:
  Run Service
  kafka:
    image: confluentinc/cp-kafka:latest
    container_name: kafka
    ports:
      - "9092:9092"
    environment:
      CLUSTER_ID: "CId-22120384"
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER        You, 7 days ago • first commit
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
      KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092,CONTROLLER://0.0.0.0:9093
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@localhost:9093
      KAFKA_LOG_DIRS: /tmp/kraft-combined-logs
```

Kafka:

- Kéo confluentinc/cp-kafka phiên bản mới nhất(4.0.0) của kafka về, chọn port mặc định là 9092 để mapping.
- Phiên bản 4.0.0 đặc biệt ở chỗ là nó đã bỏ đi zookeeper nên ta chỉ sử dụng mode kraft.

```yaml
  mongodb:
    image: mongo:latest
    container_name: mongodb
    ports:
      - "27017:27017"
    volumes:
      - mongodb_data:/data/db
```

Mongodb:

- Kéo image mới nhất của mongodb về, chọn port mặc định 27017 để mapping.
- Dữ liệu được lưu trữ trong volume mongodb_data.

```yaml
  spark:
    build:
      context: .
      dockerfile: Dockerfile.pyspark
    container_name: spark
    depends_on:
      - kafka
      - mongodb
    volumes:
      - ./src:/app
    working_dir: /app
    command: ["tail", "-f", "/dev/null"]

volumes:
  mongodb_data:
```

```
You, 29 minutes ago | 1 author (You)
FROM bitnami/spark:latest

USER root

RUN apt-get update && apt-get install -y \
    python3-pip \
    gcc \
    librdkafka-dev \
    && apt-get clean

RUN pip3 install --no-cache-dir \
    confluent-kafka \
    pymongo \
    pandas \
    numpy \
    python-dotenv \
    websocket-client \
    pymongo[srv] \
    dotenv \        You, 29 minutes ago • Uncommitted changes

COPY ./src /app
WORKDIR /app

CMD ["tail", "-f", "/dev/null"]
```

Spark:

- Build context ở thư mục hiện tại.
- Cấu hình cho spark depends on kafka và mongodb.
- Khi build thư mục src sẽ được mount vào bên trong thư mục app của Docker.
- Cấu hình chi tiết được để Dockerfile.pyspark được dùng để cài các gói cần thiết.

Kết quả sau khi build:

Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
22120384	-	-	-	2.13%	1 hour ago	
mongodb	ffbaf41c74fe	mongo:latest	27017:27017	0.67%	1 hour ago	
kafka	e567c2e54ec2	confluentinc/cp-kafka:lates	9092:9092	1.45%	1 hour ago	
spark	0702f24fe762	22120384-spark		0.01%	1 hour ago	

- Một phần đặc biệt nữa là cấu hình cho file .env:

```
# ========== Kafka ==========
KAFKA_BROKER=localhost:9092
KAFKA_TOPIC_RAW=btc-price
KAFKA_TOPIC_MOVING=btc-price-moving
KAFKA_TOPIC_ZSCORE=btc-price-zscore
KAFKA_CLIENT_ID_EXTRACT=extract-btc-price

# ========== Binance API ==========
BINANCE_SYMBOL=BTCUSDT

# ========== MongoDB ==========
MONGO_URI=mongodb://localhost:27017
MONGO_DB=crypto
MONGO_COLLECTION_PREFIX=btc-price-zscore-
```

3. Extraction

3.1. Đảm bảo tần suất

Đối với yêu cầu đầu tiên là lấy dữ liệu từ Binance về với tần suất ít nhất là 1 request/ 100 ms, em tìm được 2 giải pháp là chạy đa luồng và websocket.

Phương pháp đầu tiên là REST API kết hợp với đa luồng thuần. Đối với giải pháp này, ta giao tiếp với web bằng cách gửi request đi và nhận về response => Luôn phải tốn thời gian cho việc gửi xử lý gửi request và đợi nhận response cho mỗi lần muốn lấy dữ liệu. Chạy đa luồng chỉ đơn giản là gửi nhiều request hơn trong khoảng thời gian rất nhỏ (gần như song song).

Phương pháp thứ hai là sử dụng websocket, phương pháp này sẽ thực hiện cơ chế bắt tay. Client gửi request bắt tay đến server, server đồng ý bắt tay và sau đó cứ mỗi khi dữ liệu cập nhật server sẽ trả liên tục phản hồi về cho client. => Tối ưu hơn.

Để rõ hơn, em sẽ so sánh 2 giải pháp này trên các khía cạnh sau:

3.1.1. Độ phức tạp khi code

- Sử dụng đa luồng thuần túy yêu cầu ta xử lý cực kỳ phức tạp. Ví dụ như hình giải pháp cho đề bài để tăng hiệu suất:

```python
def fetch_price_worker(price_queue):
    while True:
        try:
            response = requests.get("https://api.binance.com/api/v3/ticker/price?symbol=BTCUSDT")
            data = response.json()
            price_queue.put(float(data['price']))
            time.sleep(0.1)
        except Exception as e:
            print(f"Error fetching price: {e}")

def kafka_producer_worker(price_queue):
    producer = create_producer()
    while True:
        try:
            price = price_queue.get(timeout=1)
            event = {
                "symbol": "BTCUSDT",
                "price": price,
                "timestamp": get_rounded_iso_time()
            }
            producer.produce(topic="btc-price", value=json.dumps(event))
            producer.poll(0)
        except queue.Empty:
            continue

def main_threading_approach():
    price_queue = queue.Queue()

    fetch_thread = threading.Thread(target=fetch_price_worker, args=(price_queue,))
    kafka_thread = threading.Thread(target=kafka_producer_worker, args=(price_queue,))

    fetch_thread.start()
    kafka_thread.start()

    fetch_thread.join()
    kafka_thread.join()
```

- Ngược lại websocket thì vô cùng đơn giản. Trong code em đã implement:

```python
def on_message(ws, message):
    global latest_price
    try:
        data = json.loads(message)
        latest_price = float(data['p'])
    except Exception as e:
        print(f"Error processing message: {e}")

def on_error(ws, error):
    print(f"WebSocket Error: {error}")

def on_close(ws, close_status_code, close_msg):
    print(f"WebSocket Closed: {close_status_code}, {close_msg}")

def on_open(ws):
    print("WebSocket connection opened.")

def start_websocket():
    ws = websocket.WebSocketApp(
        WS_URL,
        on_open=on_open,
        on_message=on_message,
        on_error=on_error,
        on_close=on_close
    )
    threading.Thread(target=ws.run_forever, daemon=True).start()

def send_message():
    global message_count, start_minute
    start_websocket()
```

3.1.2. Hiệu suất

Tốc độ gửi request và nhận về response phụ thuộc vào đường truyền mạng. Trong khi websocket chỉ gửi 1 request bắt tay lần đầu duy nhất thì REST API luôn phải gửi yêu cầu rồi mới được nhận về response

- Đa luồng + REST API.

```
Client ─── HTTP Request ──▶ Binance Server
       ◀── HTTP Response ──

• Latency: ~50-200ms (HTTP overhead)
• Bandwidth: Cao (full HTTP headers mỗi request)
• CPU Usage: Cao (connection setup/teardown)
```

- Websocket:

```
Binance Server ──────── Push Data ──────▶ Client

• Latency: ~1-5ms
• Bandwidth: Minimal (chỉ data thay đổi)
• CPU Usage: Thấp (event-driven)
```

3.1.3. Tiêu tốn tài nguyên

Như đã đề cập ở trên, websocket là kết nối ổn định 1 lần bắt tay còn REST API phải gửi nhiều connection dẫn đến tiêu tốn tài nguyên hơn rất nhiều. Ở dưới là ước tính chi phí cho cả 2:

- Websocket

```
# 1 persistent connection
# 1 background thread
# Minimal memory footprint

Resource Usage:
├── TCP Connections: 1
├── Threads: 2 (main + websocket)
├── Memory: ~10-20MB
└── CPU: ~1-2% (idle most of time)
```

- REST API + đa luồng:

```
# Multiple HTTP connections
# Multiple worker threads
# Connection pooling overhead

Resource Usage:
├── TCP Connections: 10-50 (connection pool)
├── Threads: 5-10 (workers + pool management)
├── Memory: ~50-100MB
└── CPU: ~5-10% (constant HTTP processing)
```

3.2. Chi tiết cách ingest dữ liệu vào kafka

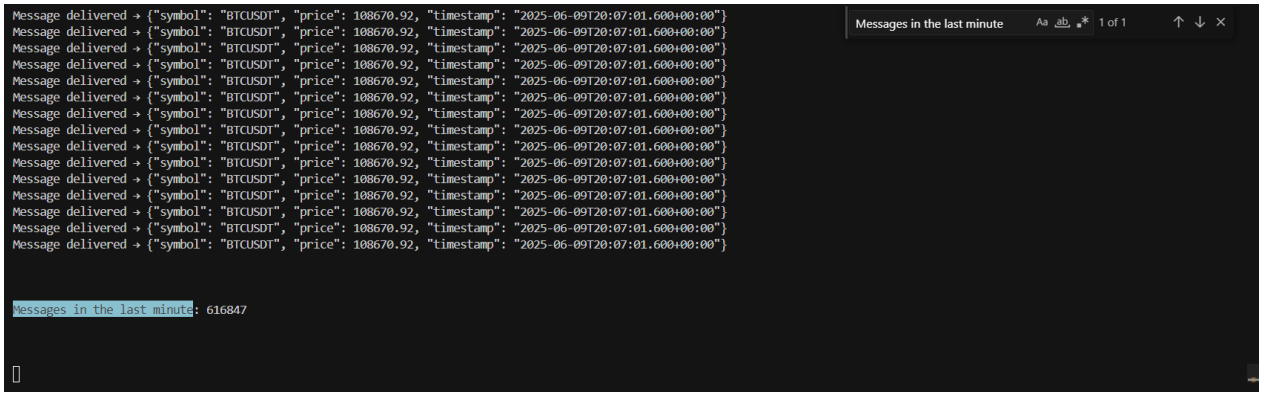
- Lấy dữ liệu từ Binance API thông qua đường dẫn trên bằng hàm start_websocket(). Ở đây ta sẽ tiến hành bắt tay. Ta cần phải cấu hình các phương thức on_... để biết được lỗi, thông báo kết nối thành công, đóng kết nối,…

```
SYMBOL = os.getenv("BINANCE_SYMBOL").lower()
WS_URL = f"wss://stream.binance.com:9443/ws/{SYMBOL}@trade"
```

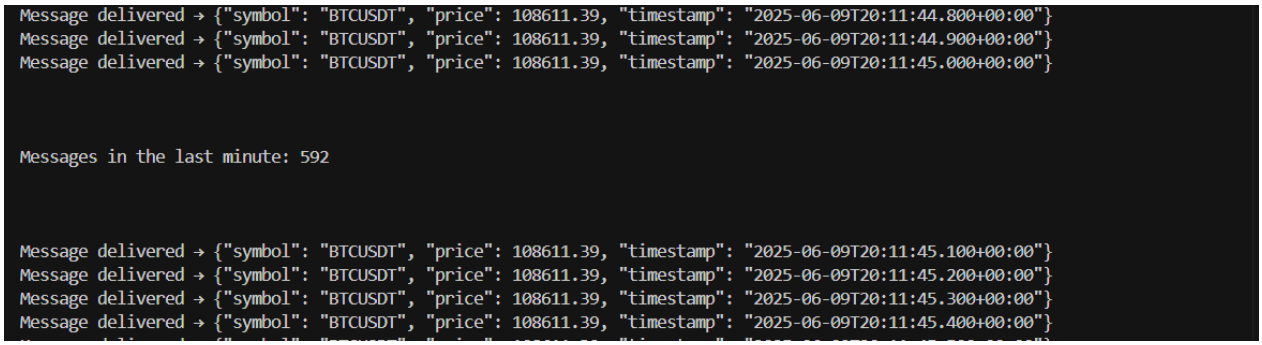
```
def start_websocket():
    ws = websocket.WebSocketApp(
        WS_URL,
        on_open=on_open,
        on_message=on_message,
        on_error=on_error,
        on_close=on_close
    )
    threading.Thread(target=ws.run_forever, daemon=True).start()
```

```
def on_message(ws, message):
    global latest_price
    try:
        data = json.loads(message)
        latest_price = float(data['p'])
    except Exception as e:
        print(f"Error processing message: {e}")

def on_error(ws, error):
    print(f"WebSocket Error: {error}")

def on_close(ws, close_status_code, close_msg):
    print(f"WebSocket Closed: {close_status_code}, {close_msg}")

def on_open(ws):
    print("WebSocket connection opened.")
```

- Tốc độ gửi kết quả về của Binance là cực kì nhanh, giả sử như ảnh dưới ta cho interval = 0. 616847 responses/phút ≈ 10 responses/ms gấp 1000 nghìn lần yêu cầu cơ bản của đề bài. Và đó còn chưa phải mức cao nhất vì ta chỉ đang chạy trên 1 node và còn chưa có mạng mạnh thật sự.



- Như vậy để khắc phục vấn đề websocket gửi quá nhanh. Ta sẽ set interval = 0.1s = 100ms. Interval này mang ý nghĩ là thời gian từ khi nhận response mới nhất của binance => xử lý dữ liệu nhận về => tạo message => rồi đẩy lên kafka topic. Việc setup như vậy sẽ đảm bảo được ta muốn tần suất gửi message lên kafka là bao nhiêu trong 100ms. Trong ảnh 1, ta sẽ thấy messages in the last minute chỉ bằng 592 < 600. Nguyên nhân của vấn đề này là do tùy thời điểm CPU sẽ có độ lệch xử lý nên sẽ có 1 số message quá 0,1s rồi mới xử lý xong. Điều này làm chậm các messages sau. Tuy nhiên diều này là không đáng kể. Với cách làm này, ta chỉ luôn lấy dữ liệu mới nhất mỗi 100ms còn những response trước đó websocket trả về ta đều bỏ qua. => Đảm bảo real-time.



- Ngoài ra còn một bước trung gian cần nói nữa đó là tạo ra timestamp. Các bước lần lượt là: Lấy thời gian hiện tại theo chuẩn UTC thay vì giờ hiện tại vì như vậy sẽ tối ưu cho spark xử lý sau này cũng như ingest data vào kafka => Tiếp theo đó, ta chuyển về ms rồi làm tròn bằng cách chia lấy nguyên cho 100 rồi lại nhân cho 100. Nguyên nhân ta làm như vậy thay vì dùng hàm round là để đảm

bảo luôn làm tròn xuống. => Chuyển lại về datetime rồi trả ra định dạng **ISO8601** như yêu cầu đề bài.

```
def get_rounded_iso_time(resolution_ms=100):
    now = datetime.now(timezone.utc)
    rounded_ts = int(now.timestamp() * 1000) // resolution_ms * resolution_ms
    rounded_dt = datetime.fromtimestamp(rounded_ts / 1000, tz=timezone.utc)

    return rounded_dt.isoformat(timespec='milliseconds')
```

- Sau khi xử lý xong, ta gửi các message lên kafka topic.

3.3. Kết quả

Websocket là lựa chọn hoàn hảo cho bài toán này, việc điều chỉnh tuần suất như mong muốn chỉ đơn giản là thiết lập interval vì websocket trả phản hồi quá nhanh rồi.

```
Message delivered → {"symbol": "BTCUSDT", "price": 109039.56, "timestamp": "2025-06-10T14:18:40.700+00:00"}
Message delivered → {"symbol": "BTCUSDT", "price": 109039.56, "timestamp": "2025-06-10T14:18:40.800+00:00"}
Message delivered → {"symbol": "BTCUSDT", "price": 109039.56, "timestamp": "2025-06-10T14:18:40.900+00:00"}
Message delivered → {"symbol": "BTCUSDT", "price": 109039.55, "timestamp": "2025-06-10T14:18:41.000+00:00"}
Message delivered → {"symbol": "BTCUSDT", "price": 109039.55, "timestamp": "2025-06-10T14:18:41.100+00:00"}
Message delivered → {"symbol": "BTCUSDT", "price": 109039.56, "timestamp": "2025-06-10T14:18:41.200+00:00"}
Message delivered → {"symbol": "BTCUSDT", "price": 109039.56, "timestamp": "2025-06-10T14:18:41.300+00:00"}
Message delivered → {"symbol": "BTCUSDT", "price": 109039.56, "timestamp": "2025-06-10T14:18:41.400+00:00"}
Message delivered → {"symbol": "BTCUSDT", "price": 109039.56, "timestamp": "2025-06-10T14:18:41.500+00:00"}
Message delivered → {"symbol": "BTCUSDT", "price": 109039.55, "timestamp": "2025-06-10T14:18:41.600+00:00"}
Message delivered → {"symbol": "BTCUSDT", "price": 109039.55, "timestamp": "2025-06-10T14:18:41.700+00:00"}
Message delivered → {"symbol": "BTCUSDT", "price": 109039.12, "timestamp": "2025-06-10T14:18:41.800+00:00"}
Message delivered → {"symbol": "BTCUSDT", "price": 109038.99, "timestamp": "2025-06-10T14:18:41.900+00:00"}



Messages in the last minute: 600



Message delivered → {"symbol": "BTCUSDT", "price": 109038.99, "timestamp": "2025-06-10T14:18:42.000+00:00"}
Message delivered → {"symbol": "BTCUSDT", "price": 109038.99, "timestamp": "2025-06-10T14:18:42.100+00:00"}
Message delivered → {"symbol": "BTCUSDT", "price": 109038.99, "timestamp": "2025-06-10T14:18:42.200+00:00"}
Message delivered → {"symbol": "BTCUSDT", "price": 109038.99, "timestamp": "2025-06-10T14:18:42.300+00:00"}
```

4. Transformation

4.1. Moving average

- Đầu tiên ta có một hàm để tạo spark session. Cấu hình bao gồm:
  - .appname = “BTCMovingStatistics”: tên spark app name.
  - .config(“spark.sql.session.timeZone”, “UTC”): cấu hình múi giờ là “UTC” để đồng bộ với Kafka và tăng hiệu suất xử lý
  - .config (“spark.hadoop.fs.defaultFS”. “file:///”): Spark luôn dùng hadoop mặc định cho việc lưu checkpoint nên ta cần cấu hình là folder checkpoints ở local(đến phần write to kafka ở dưới sẽ rõ hơn)

```python
def create_spark_session():
    spark = SparkSession.builder \
        .appName("BTCMovingStatistics") \
        .config("spark.sql.session.timeZone", "UTC") \
        .config("spark.hadoop.fs.defaultFS", "file:///") \
        .config("spark.sql.streaming.statefulOperator.checkCorrectness.enabled", "false") \
        .getOrCreate()
    spark.sparkContext.setLogLevel("ERROR")      You, 4 days ago • push bug

    return spark
```

  - .config(“spark.sql.streaming.statefulOperator.checkCorrectness.enabled”, “false”): Trong spark có khái niệm **Stateful Operations.** Chúng là những operations cần để duy trì trạng thái. Trong phần bài làm của em thì nó là khi ta group các các windows theo “symbol”. Khái niệm tiếp theo là **Correctness Checking.** Đây là quá trình validation mà spark thực hiện để đảm bảo: State store consistency(sự nhất quán trong việc lưu trữ trạng thái), Watermark Consistency(sư nhất quán trong việc gán các watermark cho dữ liệu đến trễ), Event time ordering(Thứ tự của các event). Ba hình ở dưới là ví dụ mô tả rõ phần này hơn.

```
# Spark kiểm tra xem state được lưu trữ có consistent không
# Ví dụ: Window state cho 30s window
{
  "window_start": "2024-01-15T14:23:30.000Z",
  "window_end": "2024-01-15T14:24:00.000Z",
  "symbol": "BTCUSDT",
  "count": 300,           # Số records trong window
  "sum": 7024875.5,       # Tổng giá trị để tính avg
  "sum_squares": 164525.3 # Tổng bình phương để tính stddev
}
```

```
# Kiểm tra watermark có tiến triển đúng không
previous_watermark = "2024-01-15T14:23:40.000Z"
current_watermark = "2024-01-15T14:23:50.000Z"
```

```
# Kiểm tra events có đến đúng thứ tự event-time không
events = [
    {"timestamp": "14:23:45", "price": 23400},  # OK
    {"timestamp": "14:23:47", "price": 23405},  # OK
    {"timestamp": "14:23:44", "price": 23398}   # Late event
]
```

- Vậy vì sao cần tắt tính năng này đi? Mục đích là để tăng hiệu suất. Vì em đang chạy lab này trên một máy duy nhất, dữ liệu nhận về từ binance lại quá nhanh, spark streaming chỉ đơn giản là chia việc xử lý dữ liệu vào các micro batch chứ không phải là streaming thật sự. Nếu phải tính toán xử lý quá nhiều thì sẽ cực kì chậm trong khi lab này yêu cầu có độ trễ thấp. Hình bên dưới là so sánh hiệu suất khi bật và tắt chế độ trong spark và vì sao khi bật chế độ này lên hiệu suất lại kém đi.

```
# Mỗi batch Spark phải:
# 1. Validate state store integrity
# 2. Check watermark progression
# 3. Verify event ordering
# 4. Validate aggregation consistency
# 5. Check memory usage
# 6. Verify checkpoint data
```

```
# With correctness checking ENABLED:
Batch processing time: 250ms average
Throughput: 400 records/second
Memory usage: 850MB
CPU usage: 25%

# With correctness checking DISABLED:
Batch processing time: 80ms average
Throughput: 1250 records/second
Memory usage: 650MB
CPU usage: 8%

# Performance improvement: ~3x faster
```

- Tiếp theo ta định nghĩa schema để đọc từ kafka và windows để thực hiện xử lý dữ liệu sau này:

```python
def define_windows():
    windows = [
        ("30s", "30 seconds"),
        ("1m", "1 minute"),
        ("5m", "5 minutes"),
        ("15m", "15 minutes"),
        ("30m", "30 minutes"),
        ("1h", "1 hour")]

    return windows

def define_schema():
    schema = StructType([
        StructField("symbol", StringType()),
        StructField("price", DoubleType()),
        StructField("timestamp", TimestampType())
    ])

    return schema
```

- Khi đọc dữ liệu từ kafka stream, em chú ý 2 yếu tố là startingOffsets = latest và maxOffsetsPerTrigger = 100 để luôn đọc dữ liệu mới nhất và giới hạn 100 offsets/batch để tránh việc quá tải.

```
def read_from_kafka(spark):
    df_raw = spark \
        .readStream \
        .format("kafka") \
        .option("kafka.bootstrap.servers", KAFKA_BROKER) \
        .option("subscribe", KAFKA_TOPIC_RAW) \
        .option("startingOffsets", "latest") \
        .option("maxOffsetsPerTrigger", "100") \        You, 4 da
        .load()

    return df_raw
```

- Kafka nhận/gửi dữ liệu ở dạng byte nên ta cần parse nó về từ json rồi mới sử dụng. Cấu trúc của một kafka message như ảnh dưới:

```
Raw Kafka Record:
{
  "key": "BTCUSDT",
  "value": '{"symbol":"BTCUSDT","price":23416.50,"timestamp"
  "topic": "btc-price",
  "partition": 0,
  "offset": 12345
}

↓ After parsing ↓

Parsed DataFrame:
+--------+---------+-------------------+
| symbol |   price |         timestamp |
+--------+---------+-------------------+
|BTCUSDT | 23416.5 | 2024-01-15 14:... |
+--------+---------+-------------------+
```

```python
def parse_json(df, schema):
    parsed_df = df.select(
        from_json(col("value").cast("string"), schema).alias("data")
    ).select("data.*")
    parsed_df = parsed_df.withColumn(
        "timestamp",
        to_timestamp(col("timestamp"))
    )

    return parsed_df
```

- Tiếp theo là xử lý với watermark.

```python
def process_window(parsed_df, window_name, window_duration):
    df_with_watermark = parsed_df.withWatermark("timestamp", "10 seconds")
```

- **Watermark** là thời gian tối đa mà một window chờ đợi dữ liệu đến trễ. Hình bên dưới là ví dụ rõ hơn về khái niệm này:

```python
# Giả sử current max event time = 14:24:00
# Watermark = max_event_time - watermark_delay
# Watermark = 14:24:00 - 10s = 14:23:50

# Spark sẽ:
# ✅ Chấp nhận events với timestamp >= 14:23:50
# ❌ Loại bỏ events với timestamp < 14:23:50 (quá late)
```

- Vậy tại sao ta lại cần watermark? Spark luôn duy trì state của window là mở để đợi các dữ liệu đến trễ tức là giữ lại các dữ liệu đến trước nhưng việc làm này sẽ tiêu tốn bộ nhớ nếu cứ đợi mãi như vậy sẽ gây tràn bộ nhớ.
- Sau khi tạo watermark cho df thì ta sẽ tiến hành tạo các khung cửa sổ. 2 hình dưới là ví dụ dễ hiểu về window specification.

```python
    window_spec = window(col("timestamp"), window_duration)
```

```
# Input: window(col("timestamp"), "30 seconds")
# Output: Struct với 2 fields:
{
  "start": "2024-01-15T14:23:30.000Z",  # Window start time
  "end": "2024-01-15T14:24:00.000Z"     # Window end time
}
```

```
# Với window duration = 30 seconds
# Events được group theo 30s intervals

Timeline:
14:23:30 ──────────────────── 14:24:00  (Window 1)
14:23:40 ──────────────────── 14:24:10  (Window 2)
14:23:50 ──────────────────── 14:24:20  (Window 3)

# Mỗi event có thể thuộc multiple windows!
Event at 14:23:45:
├── Thuộc Window 1 (14:23:30-14:24:00)
├── Thuộc Window 2 (14:23:40-14:24:10)
└── Thuộc Window 3 (14:23:50-14:24:20)
```

- Tiếp theo là slide interval(khoảng thời gian trượt – tạo cửa sổ mới). Với định nghĩa này, các cửa sổ sẽ overlap(chồng lên nhau).

```
slide_intervals = {
    "30 seconds": "10 seconds",     # Slide = 1/3 Window
    "1 minute": "20 seconds",       # Slide = 1/3 Window
    "5 minutes": "1 minute",        # Slide = 1/5 Window
    "15 minutes": "5 minutes",      # Slide = 1/3 Window
    "30 minutes": "10 minutes",     # Slide = 1/3 Window
    "1 hour": "20 minutes"          # Slide = 1/3 Window
}
```

- Vì sao cần overlap?
  - Để không mất thông tin quan trọng. Ảnh dưới là ví dụ cho thấy điều đó:

```
# Dữ liệu giá Bitcoin:
14:23:05 → $23,400
14:23:15 → $23,450  # Tăng $50
14:23:25 → $23,380  # Giảm $70
14:23:35 → $23,420  # Tăng $40

# Tumbling Window (30s, no overlap):
Window 1 (14:23:00-14:23:30): Chỉ thấy $23,400 → $23,450 → $
Window 2 (14:23:30-14:24:00): Chỉ thấy $23,420 → ...

# ❌ MISS: Transition từ $23,380 → $23,420 (quan trọng!)

# Sliding Window (30s, slide 10s):
Window 1 (14:23:00-14:23:30): $23,400 → $23,450 → $23,380
Window 2 (14:23:10-14:23:40): $23,450 → $23,380 → $23,420
Window 3 (14:23:20-14:23:50): $23,380 → $23,420 → ...

# ✅ CAPTURE: Tất cả transitions!
```

- Dễ dàng phát hiện xu hướng mới:

```
# Scenario: Bitcoin price trend
Time:  14:23:00  14:23:10  14:23:20  14:23:30  14:23:40  14:
Price:  $23,400  $23,450  $23,380  $23,420  $23,460  $23,440

# Tumbling Windows (30s):
Window 1 avg: ($23,400 + $23,450 + $23,380) / 3 = $23,410
Window 2 avg: ($23,420 + $23,460 + $23,440) / 3 = $23,440
# Jump: $23,410 → $23,440 (sudden change!)

# Sliding Windows (30s, slide 10s):
Window 1 (14:23:00-14:23:30): $23,410 avg
Window 2 (14:23:10-14:23:40): $23,417 avg  # Smooth transiti
Window 3 (14:23:20-14:23:50): $23,420 avg  # Gradual change
# Smooth: $23,410 → $23,417 → $23,420 (gradual!)
```

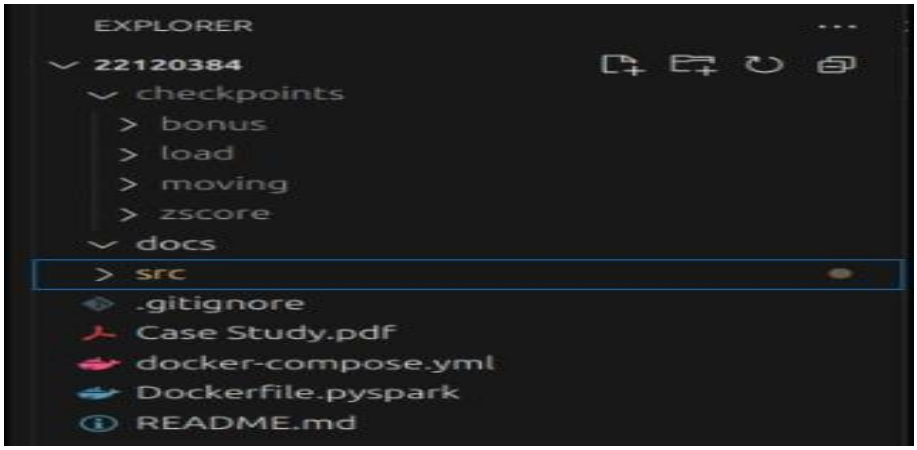
- Hàm combine_windows chỉ để kết hợp các cửa sổ lại rồi cho hàm format_output() ghi đúng định dạng ra kafka topic:

```python
def combine_windows(spark, *window_dfs):
    if not window_dfs:
        return None

    union_df = window_dfs[0]
    for df in window_dfs[1:]:
        union_df = union_df.unionAll(df)

    return union_df
```

- Bước cuối cùng là ghi dữ liệu kafka topic. Đề bài yêu cầu dùng append mode. Tuy nhiên, theo em thì update mode sẽ hợp lí hơn vì ta phải tính toán lại dữ liệu trong các khung cửa sổ mỗi khi dữ liệu mới đến. Ở đây như ta thấy thì thư mục checkpoint sẽ được tạo ra ở ../checkpoints/moving. Nếu ta đứng ở src/ thực thi file thì kết quả sẽ cho ra folder checkpoints ở local như ảnh 3.

```python
def write_to_kafka(df, checkpoint_path, topic):
    return df.writeStream \
        .format("kafka") \
        .option("kafka.bootstrap.servers", KAFKA_BROKER) \
        .option("topic", topic) \
        .option("checkpointLocation", checkpoint_path) \
        .outputMode("update") \
        .start()
```

```python
kafka_query = write_to_kafka(
    output_df,
    checkpoint_path="../checkpoints/moving",
    topic=KAFKA_TOPIC_MOVING
)
```



4.2. Zscore

- Các hàm như đọc từ kafka, ghi ra kafka, format_output cũng tương tự như trên nên em sẽ tập trung vào phần chính là hàm **calculate_zscores()**
- Hàm này sẽ tính toán zscore dựa trên btc-price và btc-price-moving

```
def calculate_zscores(price_df, stats_df):        You, yesterda
    price_df = price_df.select(
        col("timestamp").alias("price_timestamp"),
        col("symbol").alias("price_symbol"),
        col("price")
    )

    stats_df = stats_df.select(
        col("timestamp").alias("stats_timestamp"),
        col("symbol").alias("stats_symbol"),
        col("window"),
        col("avg_price"),
        col("std_price")
    )
```

- Ta sẽ join 2 df cùng timestamp lại, cùng symbol với phép inner join(chỉ lấy cái mà cả 2 bên đều có). Ví dụ minh họa như hình:

```
joined_df = price_df.join(
    stats_df,
    (price_df.price_timestamp == stats_df.stats_timestamp) &
    (price_df.price_symbol == stats_df.stats_symbol),
    "inner"
)
```

```
# Before Join:
Price DF:
+-------------------+--------+-------+
|  price_timestamp  | symbol | price |
+-------------------+--------+-------+
|2024-01-15 14:23:30|BTCUSDT |23450.0|
+-------------------+--------+-------+

Stats DF:
+-------------------+--------+------+---------+---------+
|  stats_timestamp  | symbol |window|avg_price|std_price|
+-------------------+--------+------+---------+---------+
|2024-01-15 14:23:30|BTCUSDT |  30s |23420.5  |   15.2  |
|2024-01-15 14:23:30|BTCUSDT |   1m |23415.2  |   18.7  |
|2024-01-15 14:23:30|BTCUSDT |   5m |23408.1  |   22.4  |
+-------------------+--------+------+---------+---------+

# After Join:
+-------------------+--------+-------+------+---------+---------+
|     timestamp     | symbol | price |window|avg_price|std_price|
+-------------------+--------+-------+------+---------+---------+
|2024-01-15 14:23:30|BTCUSDT |23450.0|  30s |23420.5  |   15.2  |
|2024-01-15 14:23:30|BTCUSDT |23450.0|   1m |23415.2  |   18.7  |
|2024-01-15 14:23:30|BTCUSDT |23450.0|   5m |23408.1  |   22.4  |
+-------------------+--------+-------+------+---------+---------+
```

- Tính toán zscore dựa trên công thức. Ở đây em lọc col("std_price") > 0.0001 để tránh các std_price quá nhỏ, nếu gặp trường hợp này thì sẽ trả về 0.0. Công thức sẽ là (price - avg_price) / std_price. Ví dụ cho quá trình này như ảnh 2.

```
zscore_df = joined_df.withColumn(
    "zscore_price",
    when(
        (col("std_price") > 0.0001),
        (col("price") - col("avg_price")) / col("std_price")
    ).otherwise(lit(0.0))
).select(
    col("price_timestamp").alias("timestamp"),
    col("price_symbol").alias("symbol"),
    col("window"),
    col("zscore_price")
)
```

```
# Case 1: Normal calculation
price = 23450.0
avg_price = 23420.5
std_price = 15.2
zscore = (23450.0 - 23420.5) / 15.2 = 29.5 / 15.2 = 1.94

# Case 2: Low volatility (safety check)
price = 23450.0
avg_price = 23449.9
std_price = 0.00005  # Very small std
zscore = 0.0  # Fallback value

# Case 3: High volatility
price = 23450.0
avg_price = 23420.5
std_price = 50.0
zscore = (23450.0 - 23420.5) / 50.0 = 0.59  # Lower z-score
```

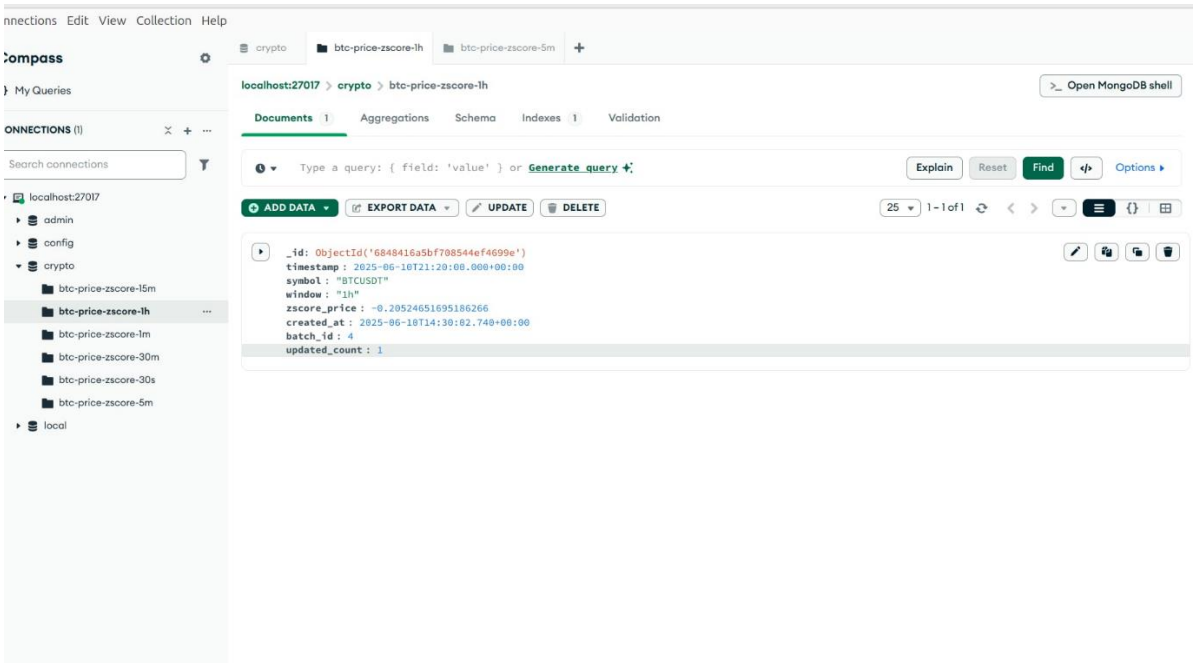
4.3. Kết quả

- Khi chạy file thực thi của 22120384_moving.py ở folder src/ sẽ cho kết quả như ảnh dưới(sẽ hơi khó nhìn do output quá dài so với màn hình console):

```
-------------------------------------------+
|timestamp          |symbol |windows

                                          |value



                                                |
+-------------------+-------+-----------------------------------------------
```

5. Loading

5.1. Schema lựa chọn



- 3 trường được thêm là id(key của mongodb), created_at(lưu lại thời điểm tạo ra) do chuẩn ban đầu là UTC cũng như spark còn tốn thời gian để xử lý dữ liệu, updated_count(số lần được update/insert sẽ được giải thích rõ hơn ở dưới)

5.2. Cách làm

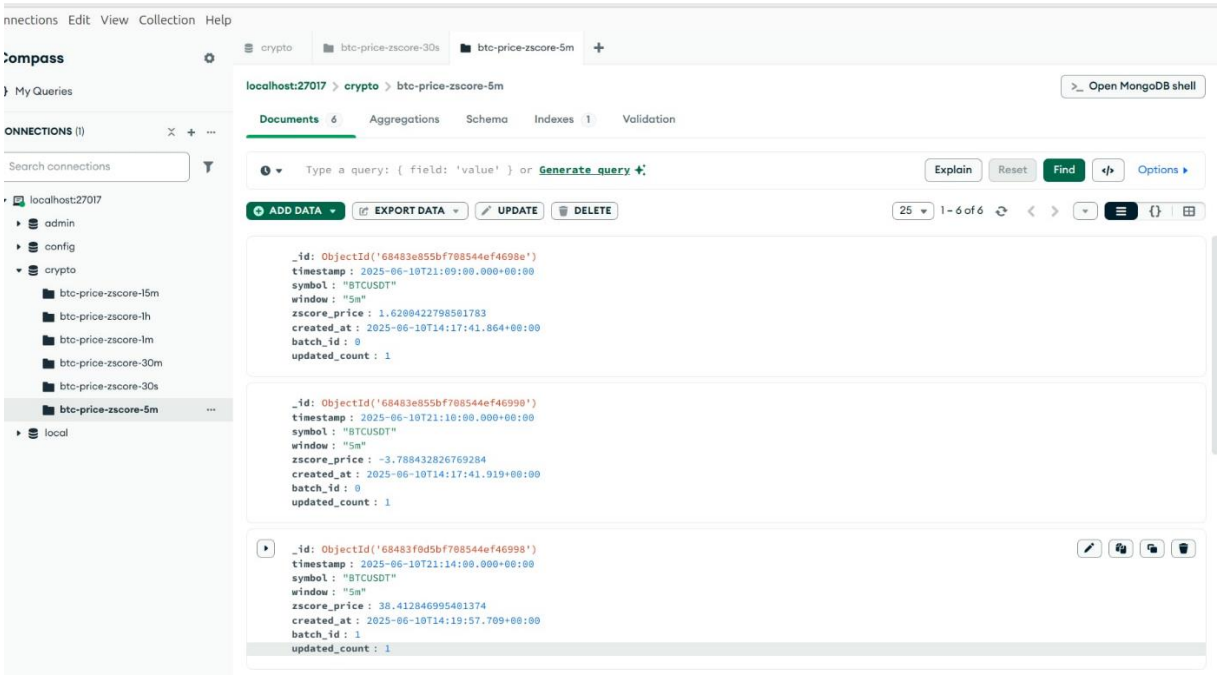
- Trong kafka, dữ liệu được ghi vào ở dạng append-only, trong khi dữ liệu đề bài được yêu cầu là lưu trữ nhiều window trong một message. Ví dụ ta có trường hợp sau:

```
{
  "timestamp": "2024-01-15T14:23:30.000Z",
  "symbol": "BTCUSDT",
  "zscores": [
    {"window": "30s", "zscore_price": 1.94},
    {"window": "1m", "zscore_price": 1.86},
  ]
},
{
  "timestamp": "2024-01-15T14:23:30.000Z",
```

"symbol": "BTCUSDT",
"zscores": [
{"window": "30s", "zscore_price": 1.94},
{"window": "1m", "zscore_price": 1.86},
{"window": "5m", "zscore_price": 1.87},
{"window": "15m", "zscore_price": 1.92},
{"window": "30m", "zscore_price": 1.89},
{"window": "1h", "zscore_price": 1.85}
]
}

- Như ta thấy cùng 1 timestamp nhưng có 2 window 30s và 1m được ghi lại. Nguyên nhân như: spark xử lý dữ liệu bằng cách chia nhỏ thành các micro batch, kết quả thứ nhất là khi batch 1 xử lý xong cho 2 window 30s và 1m, lúc sau batch 2 xử lý xong cho các window còn lại. Kafka chỉ ghi dữ liệu dạng append-only. Chính vì vậy, vấn đề trùng lắp dữ liệu trên sẽ xảy ra. Việc lưu trữ này sẽ hợp lý vì còn tùy vào nhiều mục đích dùng dữ liệu chứ không phải chỉ phục vụ mỗi việc loading vào mongodb. Cho nên thay vì xử lý vấn đề trùng lắp này ở bước transformation, ta sẽ xử lý nó ở giai đoạn loading.
- Phương pháp xử lý là chọn mode upsert(update+insert) của mongodb. Ta dựa vào timestamp và window để biết cửa sổ của timestamp đã được ghi hay chưa nếu chưa thì ta insert, updated_count = 1, nếu rồi thì ta update rồi updated_count += 1.

5.3. Kết quả


nnections Edit View Collection Help
Compass
My Queries
ONNECTIONS (1)
Search connections
localhost:27017
admin
config
crypto
btc-price-zscore-15m
btc-price-zscore-1h
btc-price-zscore-1m
btc-price-zscore-30m
btc-price-zscore-30s
btc-price-zscore-5m
local
crypto
btc-price-zscore-30s
btc-price-zscore-5m
localhost:27017 > crypto > btc-price-zscore-5m
Open MongoDB shell
Documents 6
Aggregations
Schema
Indexes 1
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
ADD DATA
EXPORT DATA
UPDATE
DELETE
25
1-6 of 6
_id: ObjectId('68483e855bf708544ef4698e')
timestamp : 2025-06-10T21:09:00.000+00:00
symbol : "BTCUSDT"
window : "5m"
zscore_price : 1.6200422798501783
created_at : 2025-06-10T14:17:41.864+00:00
batch_id : 0
updated_count : 1
_id: ObjectId('68483e855bf708544ef46990')
timestamp : 2025-06-10T21:10:00.000+00:00
symbol : "BTCUSDT"
window : "5m"
zscore_price : -3.788432826769284
created_at : 2025-06-10T14:17:41.919+00:00
batch_id : 0
updated_count : 1
_id: ObjectId('68483f0d5bf708544ef46998')
timestamp : 2025-06-10T21:14:00.000+00:00
symbol : "BTCUSDT"
window : "5m"
zscore_price : 38.412846995401374
created_at : 2025-06-10T14:19:57.709+00:00
batch_id : 1
updated_count : 1